ভারতীয় প্রযুক্তিবিদ্যা প্রতিষ্ঠান খড়্গপুর
भारतीय प्रौद्योगिकी संस्थान खड़गपुर
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

# Transformers and Agentic AI Systems


Dr. Animesh Chaturvedi

Assistant Professor: **IIT Kharagpur**

Young Researcher: **Heidelberg Laureate Forum**

and **Pingala Interaction in Computing**

Young Scientist: **Lindau Nobel Laureate Meetings**

Postdoc: **King's College London** & **The Alan Turing Institute**

PhD: **IIT Indore** MTech: **IIITDM Jabalpur**

# Introduction and Foundations

# The Central Question

## What happens when AI moves from answering questions to pursuing goals?

**Generative AI**

A user provides a prompt and the model generates a response, typically within a single interaction.

**Agentic AI**

A user provides an objective and the system can decide what information, reasoning, tools, and actions are required to accomplish that objective.

**The fundamental transition is from response generation to goal-directed action.**

# Evolution of Artificial Intelligence

1. **Traditional AI:** explicit rules and predefined logic.
2. **Machine Learning:** patterns learned from data for prediction and decision support.
3. **Deep Learning:** learned representations for complex data.
4. **Generative AI:** creation of text, images, code, and other content.
5. **Agentic AI:** goal-directed systems that combine models with tools, actions, and feedback.

**Interpretation**

These paradigms are not strict replacements for one another. Agentic systems build on existing AI capabilities and add orchestration, tool use, and feedback-driven action.

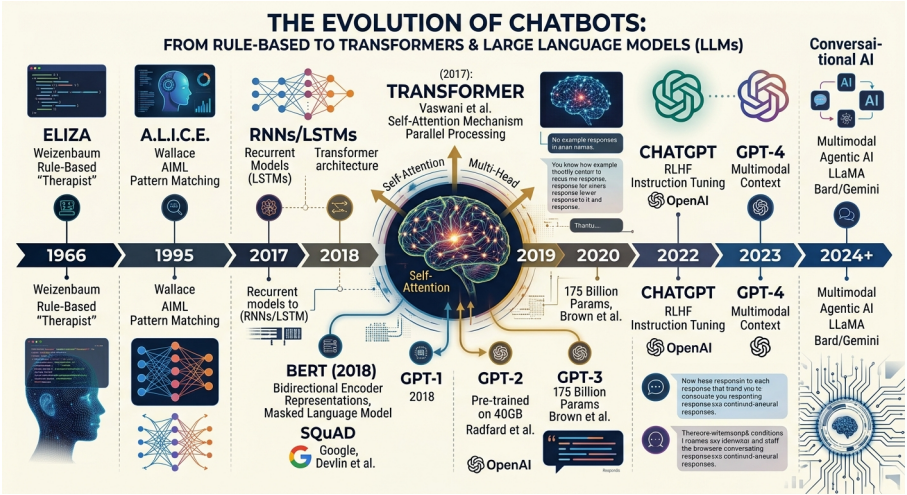
# Traditional AI vs Generative AI vs Agentic AI

| Paradigm | Primary behaviour | Typical interaction |
|---|---|---|
| Traditional AI | Applies predefined rules or learned decision functions. | Input → prediction/decision |
| Machine Learning | Learns patterns from data to make predictions. | Data → model → prediction |
| Generative AI | Generates new content from a prompt or context. | Prompt → generated content |
| Agentic AI | Pursues an objective using reasoning, tools, actions, and feedback. | Goal → plan → act → observe → adapt |

**Agentic AI adds an action-oriented control loop around AI models.**

# Evolution of ChatBots

Rules, Patterns, RNN, LSTM, **Transformers, BERT, GPT**, Multimodal and Agentic AI



# Sequence Modelling with Attention

# The Sequence Modelling Problem

Consider:

**"The animal did not cross the road because it was tired."**

What does "it" refer to?

The model needs to connect information across different positions in the sentence.

**Language understanding often requires relationships between tokens that may be far apart in the sequence.**

# The Key Idea of Attention

Instead of compressing the entire previous sequence into one hidden state, attention allows a model to ask:

**"Which other tokens are relevant to the token I am processing?"**

For example:

"The cat sat on the mat because it was comfortable."

The representation of "it" can assign high attention to "cat".

**Attention creates direct information pathways between sequence elements.**

# Attention as Information Retrieval

A useful intuition is to think of attention as a retrieval mechanism.

- The current token creates a **query**.
- Other tokens provide **keys**.
- Their associated information is stored in **values**.
- Similarity between a query and keys determines which values should receive more weight.

"I love you",
'I' is aligned with 'je', 'love' is aligned with 'aime', and 'you' is aligned with 't'
An English-French alignment matrix:

| | I | love | you |
|---|---|---|---|
| je | 0.94 | 0.02 | 0.04 |
| t' | 0.11 | 0.01 | 0.88 |
| aime | 0.03 | 0.95 | 0.02 |

# Query, Key and Value: An Analogy

Imagine entering a library.

- **Query:** What information am I looking for?
- **Key:** What does each book/topic contain?
- **Value:** What information will I actually retrieve?

The query is compared against keys.
The resulting relevance scores determine how much information is retrieved from each value.
Therefore:

$$\boxed{\text{Query} + \text{Keys} \rightarrow \text{Attention Weights} \rightarrow \text{Values}}$$

**Attention is therefore a learned, content-dependent information retrieval mechanism.**

# Transformer with Attention

# Why Do We Need Transformers?

Many AI problems involve sequences:

- Natural language: words form sentences.
- Time series: observations occur over time.
- Speech: acoustic signals evolve sequentially.
- Source code: tokens form structured programs.
- DNA and proteins: biological sequences contain long-range dependencies.

**The central challenge is understanding relationships between elements of a sequence.**

# Ashish Vaswani, et al. "Attention Is All You Need" NeurIPS 2017.

## Attention Is All You Need


**Ashish Vaswani*** Google Brain avaswani@google.com

**Noam Shazeer*** Google Brain noam@google.com

**Niki Parmar*** Google Research nikip@google.com

**Jakob Uszkoreit*** Google Research uszr@google.com

**Llion Jones*** Google Research llion@google.com

**Aidan N. Gomez*** † University of Toronto aidan@cs.toronto.edu

**Łukasz Kaiser*** Google Brain lukaszkaiser@google.com

**Illia Polosukhin*** ‡ illia.polosukhin@gmail.com


### Abstract


The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.


## Ashish Vaswani

Article Talk

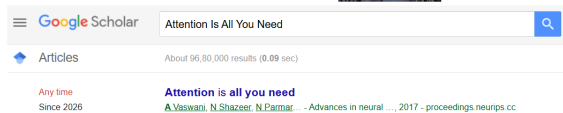
From Wikipedia, the free encyclopedia

**Ashish Vaswani** is an Indian computer scientist and entrepreneur. He conducted research at Google Brain, co-founded Adept AI, and co-founder and chief executive officer of Essential AI.[2][3][4] As of June 2026, Vaswani, along with Essential AI, was acqui-hired by Nvidia.[1]

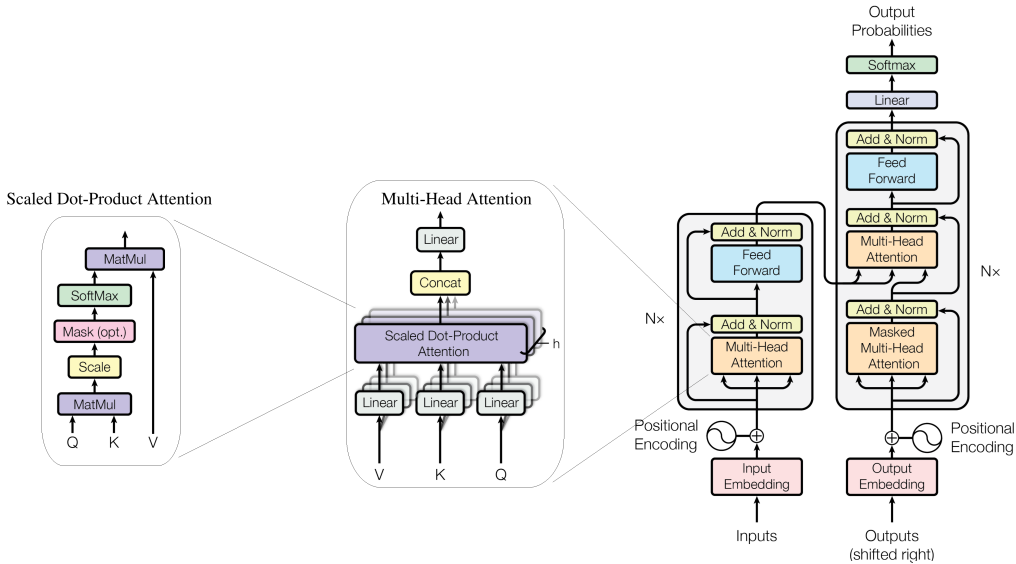
Vaswani is a co-author of the 2017 paper "Attention Is All You Need", which introduced the Transformer neural network architecture.[5] The Transformer model has been used in the development of subsequent NLP models BERT, ChatGPT, and their successors.



| Ashish Vaswani | |
|---|---|
| **Born** | 1986 (age 39–40) India |
| **Education** | University of Southern California (PhD) Birla Institute of Technology, Mesra (B.Tech.) |
| **Known for** | Transformer (deep learning architecture) |
| | **Scientific career** |
| **Fields** | Natural Language Processing Deep Learning Artificial Intelligence |
| **Workplaces** | Google Brain (2016-2021) Nvidia (2026-present)[1] |
| **Thesis** | *Smaller, Faster, and Accurate Models for Statistical Machine Translation* (2014) |
| **Doctoral advisor** | David Chiang Liang Huang |



# Transformer - model architecture



# Three Important Transformer Families

| Architecture | Main idea | Typical use |
|---|---|---|
| Encoder-only | Bidirectional contextual representation. | Classification, retrieval, representation learning |
| Decoder-only | Causal autoregressive generation. | Text generation and LLMs |
| Encoder-decoder | Input encoding followed by conditional generation. | Translation, summarization, sequence-to-sequence tasks |

**Modern LLMs are predominantly based on decoder-only Transformer variants.**

# Why Is Position Important?

Attention by itself does not inherently know whether:

"Dog bites man"

is different from:

"Man bites dog."

Both contain the same tokens but in different orders.

**Therefore, Transformers need positional information.**

# Transformers and Modern LLMs

A modern LLM can be viewed conceptually as:

$$\boxed{\textit{Tokenizer} + \textit{Embeddings} + \textit{Transformer Blocks} + \textit{Language Model Head}}$$

The Transformer provides the computational architecture.
Training provides the learned parameters.
Data provides the statistical experience.
Post-training methods shape how the model behaves.

**The Transformer architecture is the foundation, but an LLM is much more than the architecture alone.**

**LLM Pipeline:** A simplified decoder-only Transformer pipeline is:




The next token is appended to the sequence and the process repeats.

# Understanding Transformers for LLMs

1. **Tokenization:** Input text is divided into tokens that can be represented numerically.
2. **Embeddings:** Each token is mapped to a high-dimensional vector that represents learned features of the token.
3. **Attention:** Tokens exchange information so that their representations can incorporate relevant context from other positions.
4. **Feed-forward layers:** Each token representation undergoes further nonlinear transformation.
5. **Repeated Transformer blocks:** Attention and feed-forward operations progressively refine token representations.
6. **Next-token prediction:** The final representation is used to produce scores and probabilities for possible next tokens.

**Observe:** The representation of a token changes as it passes through the network. The model converts its final representation into a prediction of the next token.

# LLM Flow

Remember the following sequence:

**Text $\rightarrow$ Tokens**

$\downarrow$

**Embeddings + Position $\rightarrow$ Self-Attention**

$\downarrow$

**Feed-Forward Transformation**

$\downarrow$

**Repeated Transformer Blocks**

$\downarrow$

**Output Distribution / Prediction**

# From Text to Tokens

Transformers generally operate on tokens rather than raw words.
For example:

$$\texttt{Transformers are powerful}$$

may be decomposed into token units such as:

$$[\texttt{Transform}, \texttt{ers}, \ \texttt{are}, \ \texttt{powerful}]$$

The exact tokenization depends on the tokenizer.
The pipeline becomes:

$$\text{Text} \rightarrow \text{Tokens} \rightarrow \text{Token IDs} \rightarrow \text{Embeddings} \rightarrow \text{Transformer}$$

# Embedding in an LLM

An embedding represents a token, sentence, document, or other input as a numerical vector that a machine-learning model can process.

**Example:**

```
"Search research papers on software evolution"
```

An embedding model transforms the input into a vector:

$$\mathbf{x} = \begin{bmatrix} 0.21 & -0.34 & 0.78 & 0.15 & \cdots \end{bmatrix}$$

- Each number is a component of the embedding vector.
- The vector has a fixed number of dimensions for a given model.
- The dimensions collectively encode learned patterns of meaning and other useful features.

## What Does Embedding Dimension Mean?

If an embedding contains $d$ numerical components, its dimension is $d$. For example, a vector with 768 components has 768 dimensions.

# How Does Attention Measure Similarity?

**Goal:** Determine which tokens are most relevant to a given token.
Suppose the sentence is:

***"The animal did not cross the road because it was tired."***

To interpret the word **"it"**, the model should identify relevant contextual words, "animal".
**Attention compares a query vector from one token with key vectors from other tokens to compute relevance scores.**

$$S = QK^T$$

- $Q$: Query vectors – what each token is looking for.
- $K$: Key vectors – what information each token offers.
- $K^T$: Transpose of the key matrix.
- $S$: Matrix of query–key dot-product scores.

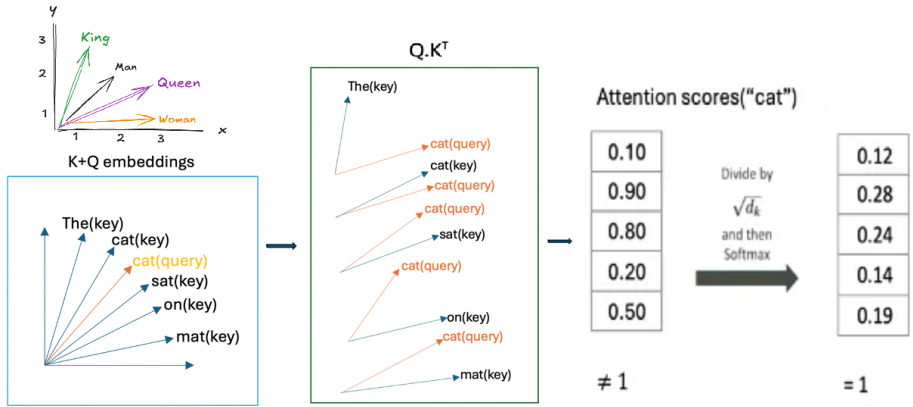
**Important:** These scores measure alignment, not physical distance.

# Compute Query-Key Similarity

**Connection to Attention:** If both vectors are normalized to unit length, then

$$\|Q\| = \|K\| = 1 \quad \Longrightarrow \quad Q \cdot K = \cos(\theta).$$

Otherwise, the dot product also depends on vector magnitudes.



# Putting It Together: Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1. **Compare:** Compute $QK^T$ to measure query–key alignment.
2. **Scale:** Divide scores by $\sqrt{d_k}$.
3. **Normalize:** Apply softmax to obtain attention weights.
4. **Aggregate:** Multiply the weights by $V$ to combine information from value vectors.

**Remember**

$QK^T$ uses dot products; it does not automatically compute cosine similarity.

Cosine similarity is obtained when query and key vectors are normalized.

Attention weights indicate how information is combined; they are not necessarily calibrated measures of semantic similarity.

# Multi-Head Attention Intuition

Imagine reading:

**"The student who studied hard passed the examination."**

Different heads may learn to emphasize different relationships:

- Head 1: subject relationships.
- Head 2: grammatical dependencies.
- Head 3: semantic associations.
- Head 4: nearby contextual relationships.

These are illustrative interpretations rather than guaranteed semantic roles of individual heads.

# Why Transformers Scale So Well

Transformers provide several advantages:

- Attention allows direct interactions between distant tokens.
- Training can be highly parallelized across sequence positions.
- Large datasets and large models can be efficiently exploited.
- The architecture is highly compositional and stackable.
- The same basic architecture can support language, vision, audio, code, and multimodal systems.

**Scalability is one of the defining strengths of Transformers.**

# Transformers Beyond NLP

The Transformer architecture has been applied to:

- Natural language processing
- Computer vision
- Speech processing
- Time-series forecasting
- Code generation
- Protein modeling
- Multimodal learning
- Recommendation systems

**The underlying idea is general: learn relationships among elements through attention.**

# Agentic AI

# What Is an AI Agent?

An *AI agent* is a software system that perceives information about its environment, reasons about a goal, selects actions, uses available tools, observes the consequences, and iteratively adapts its behaviour.

A useful abstraction is:

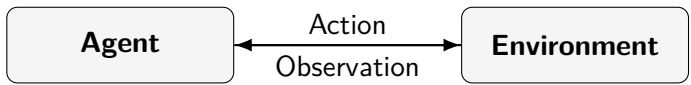
**Goal + State + Reasoning + Tools + Action + Feedback**

An LLM can be the reasoning component of an agent, but an LLM by itself is not necessarily an agent.

# A Rational Agent Perspective

A classical AI agent can be viewed as a system that:

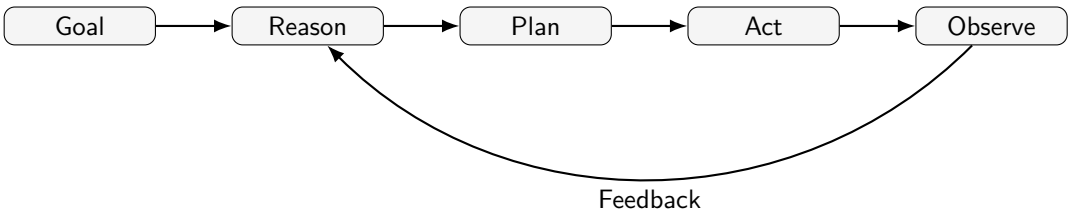
- receives observations from an environment,
- maintains information about the current state,
- chooses an action,
- receives new observations,
- and continues until a task or objective is achieved.




Agentic AI extends this classical idea by using foundation models, external tools, memory, planning mechanisms, and modern software infrastructure.

# The Agentic AI Loop

**Agentic AI** refers to AI systems that can reason about a goal, plan actions, invoke tools, observe results, and revise their actions until a task is completed or requires human intervention.




**Goal → Reason → Plan → Act → Observe → Re-plan**

# Why the Feedback Loop Matters

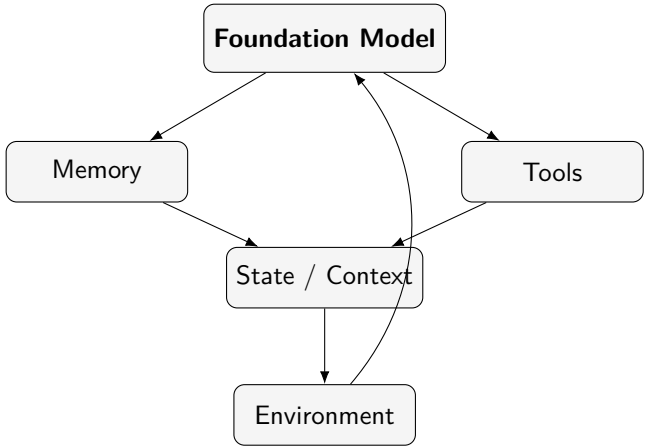
Consider an agent asked to analyse a company's sales performance.

1. The agent identifies the required data.
2. It retrieves the data from a database.
3. It performs analysis.
4. It discovers that one required variable is missing.
5. It changes its plan and retrieves another data source.
6. It continues the analysis.

**Key Difference**

A fixed workflow follows a predefined sequence. An agent can potentially modify its next action based on what happened in the previous action.

# Core Components of an Agent



# Perception: What Does the Agent Know?

An agent may obtain information from many sources:

- Natural-language instructions from a human.
- Documents and knowledge bases.
- Databases and structured enterprise data.
- Web pages and external APIs.
- Images, audio, video, and other modalities.
- System logs, sensors, or application state.

Perception is the process of transforming environmental information into a representation that the agent can reason about.

# Reasoning and Decision Making

The agent must answer questions such as:

- What is the actual objective?
- What information is currently available?
- What information is missing?
- What action should be performed next?
- Which tool is appropriate?
- Did the previous action produce the expected result?
- Should the current plan be changed?

**Reasoning converts observations and goals into decisions.**

# Planning: From Goal to Subtasks

Suppose the goal is:

**"Prepare a market-entry report for an AI product."**

The agent may decompose it into:

1. Understand the target market.
2. Identify major competitors.
3. Collect market and competitor data.
4. Analyse customer segments.
5. Estimate opportunities and risks.
6. Develop strategic recommendations.
7. Produce the final report.

Planning transforms a high-level objective into a sequence or hierarchy of executable subtasks.

# Planning Strategies

**Sequential planning**

$$Task \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow Result$$

**Hierarchical planning**

$$Goal \rightarrow Subgoals \rightarrow Tasks \rightarrow Actions$$

**Dynamic planning**

$$Plan \rightarrow Action \rightarrow Observation \rightarrow Re-plan \rightarrow Action$$

Dynamic planning is particularly useful when the environment is uncertain or when the result of one action changes what should happen next.

# Memory in Agentic AI

Memory allows an agent to maintain useful information beyond the immediate model output.

- **Working memory:** information required for the current task.
- **Long-term memory:** persistent information retained across tasks.
- **Episodic memory:** records of previous experiences or interactions.
- **Semantic memory:** facts and structured knowledge.

Memory allows an agent to behave consistently across multiple steps and, in some systems, across multiple sessions.

# Tools: Giving AI the Ability to Act

An LLM normally produces text.
An agent can use tools such as:

- Web search
- Databases
- Python
- Calculators
- APIs
- Code repositories
- Cloud services
- Email
- Calendars
- Enterprise applications

**Tools connect the reasoning capability of an AI model to the external world.**

# Tool Calling / Function Calling

The model does not need to perform every operation itself.
Instead, it can decide:

**"I need to invoke this tool with these parameters."**

Example:

1. User: "Find tomorrow's available meeting slots."
2. Agent determines that calendar information is required.
3. Agent calls the calendar tool.
4. Calendar returns available slots.
5. Agent interprets the result and responds.

Tool calling separates language-based decision making from the actual execution of external operations.

# The ReAct Concept

A widely discussed agent pattern interleaves reasoning and action:

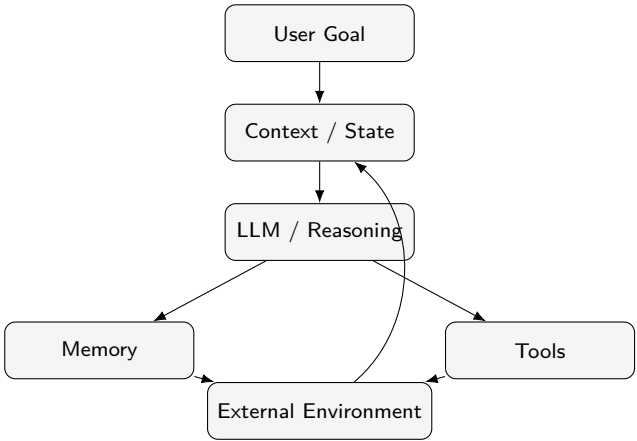
$$\textbf{Reason} \rightarrow \textbf{Act} \rightarrow \textbf{Observe}$$

Example:

- The agent identifies that current information is insufficient.
- It chooses a search tool.
- The search result becomes a new observation.
- The agent determines the next required action.
- The cycle continues until sufficient evidence is available.

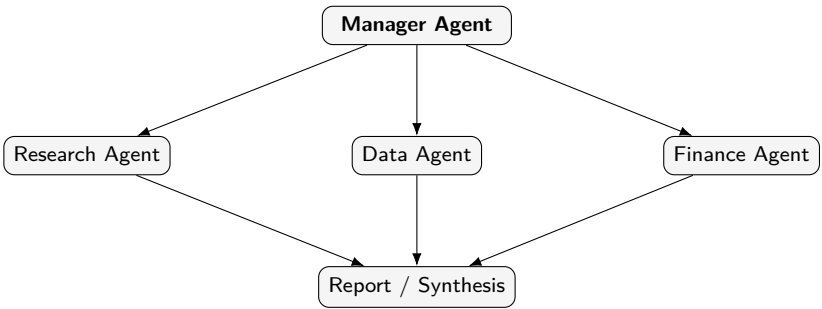
The important idea is not a particular prompting format; it is the repeated interaction between decision making and environmental feedback.

# A Complete Single-Agent Architecture



# From One Agent to Multiple Agents

A complex problem can be decomposed among specialised agents.



# Architectures and Coordination

# Why Use Multi-Agent Systems?

Different agents can specialize in different responsibilities.

- **Specialization:** each agent can focus on a particular task or domain.
- **Parallelism:** independent tasks can potentially be executed simultaneously.
- **Modularity:** individual agents can be modified without redesigning the entire system.
- **Debate:** multiple agents can produce and critique alternative solutions.
- **Coordination:** a manager agent can combine outputs from specialized agents.

Multi-agent systems introduce a new problem: the system must coordinate not only actions, but also the agents that decide those actions.

# Agent Coordination

Multi-agent systems can involve:

- **Collaboration:** agents pursue a shared objective.
- **Delegation:** one agent assigns subtasks to another.
- **Negotiation:** agents resolve conflicting preferences or resources.
- **Competition:** agents generate competing solutions.
- **Consensus:** agents combine multiple perspectives into a final decision.

**This connects Agentic AI with distributed AI, multi-agent systems, game theory, and organizational decision making.**

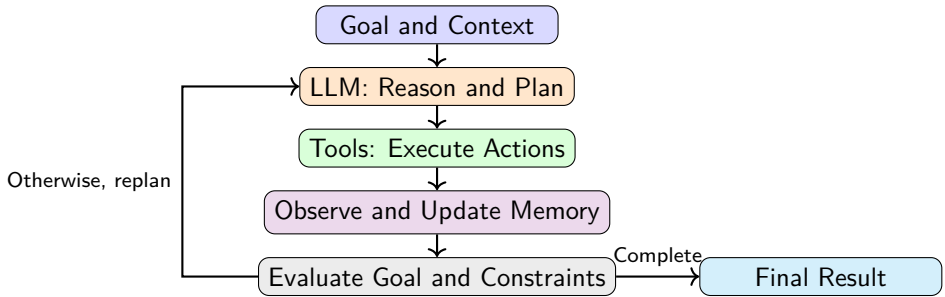
# Agentic AI vs Reinforcement Learning

These concepts are related but not identical.

| **Reinforcement Learning** | **LLM-based Agent** |
|---|---|
| State $\rightarrow$ Action $\rightarrow$ Reward | Goal $\rightarrow$ Reason $\rightarrow$ Action $\rightarrow$ Observation |
| Policy is often learned from interaction. | Decision making may be driven by a foundation model and orchestration logic. |
| Reward provides a central learning signal. | Success criteria can include task completion, safety, quality, and human feedback. |

**An LLM agent does not have to be trained using reinforcement learning.**

# How the Technologies Work Together




**Takeaway**

Agentic behavior emerges from the interaction of models, planning, tools, memory, feedback, and execution controls—not from the LLM alone.

# Applications and Case Study

# Business Applications of Agentic AI

**Customer Service**

- Understand customer request
- Retrieve account information
- Apply policies
- Execute approved actions

**Marketing**

- Analyse campaigns
- Identify customer segments
- Generate content
- Monitor outcomes

**Finance**

- Analyse reports
- Detect anomalies
- Forecast trends
- Prepare management reports

**Supply Chain**

- Monitor inventory
- Analyse demand
- Identify disruptions
- Recommend corrective actions

# Agentic AI in Data Science

A data science agent could transform:

**Business Question**

into:

$$\textit{DataDiscovery} \rightarrow \textit{Cleaning} \rightarrow \textit{EDA} \rightarrow \textit{FeatureEngineering} \rightarrow \textit{Modeling}$$

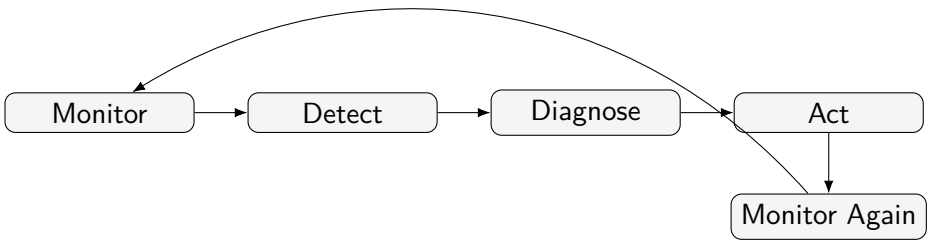
$$\textit{Modeling} \rightarrow \textit{Evaluation} \rightarrow \textit{Interpretation} \rightarrow \textit{Report}$$

The agent may select tools dynamically depending on intermediate results.

The critical research question is not whether an agent can execute these steps, but whether it can reliably determine that its analysis and conclusions are correct.

# Agentic AI and Cloud Computing

An agent can operate as an intelligent control layer over cloud infrastructure.




Applications include autoscaling, fault management, resource optimization, cost optimization, and AIOps.

# Case Study: Market Entry Strategy

## Goal: Develop a market-entry strategy for an EV company.

An agentic system could:

1. Research the target market.
2. Identify competitors.
3. Retrieve government and industry information.
4. Analyse customer segments.
5. Estimate market opportunities.
6. Analyse competitors' pricing.
7. Develop alternative strategies.
8. Compare risks and expected outcomes.
9. Produce an executive report.

**The agent is not simply writing the report; it is orchestrating the work required to produce the report.**

# Case Study: Market Entry Strategy

Where Can It Fail?
To identify possible failure points.

- What if the market data is outdated?
- What if a search result contains misleading information?
- What if the agent misunderstands the business objective?
- What if two agents provide contradictory conclusions?
- What if the financial assumptions are incorrect?
- What if the agent takes an unauthorized action?
- Who should approve the final strategic recommendation?

**Capability is only one dimension; reliability and governance are equally important.**

# Reliability, Safety, and Evaluation

# Human-in-the-Loop

Autonomy should be proportional to the consequences of an action.

| Risk level | Recommended control |
|---|---|
| Low | Automate routine, reversible actions. |
| Medium | Execute with monitoring and clear escalation rules. |
| High | Require explicit human approval before execution. |

High-risk examples include financial transactions, deletion of important data, production infrastructure changes, and decisions affecting individuals.

# Major Challenge: Hallucination

A conventional hallucination produces:

**Incorrect information**

An agentic hallucination can produce:

Incorrect belief $\rightarrow$ Incorrect plan $\rightarrow$ Incorrect action $\rightarrow$ Real-world consequence

**Agentic systems amplify the consequences of unreliable reasoning because their outputs can trigger actions.**

# Security Challenges

Giving an agent access to tools creates a larger attack surface.
Important risks include:

- **Prompt injection:** malicious instructions influence the agent's behaviour.
- **Data leakage:** sensitive information may be exposed to inappropriate tools or users.
- **Unauthorized actions:** the agent may invoke tools beyond intended permissions.
- **Credential exposure:** tools may provide access to sensitive credentials or services.
- **Excessive autonomy:** an error can become an operational incident.

**Agent security must protect the entire action chain, not only the language model.**

# Cascading Errors

Agentic systems can create chains of dependent decisions.

$$E_1 \rightarrow D_1 \rightarrow A_1 \rightarrow E_2 \rightarrow D_2 \rightarrow A_2 \rightarrow \cdots$$

where:

- $E$ represents an observation or environmental input,
- $D$ represents a decision,
- $A$ represents an action.

A small error early in an autonomous workflow can propagate through later decisions, making error detection and recovery essential.

# How Do We Evaluate an Agent?

Traditional ML metrics are not sufficient for many agentic tasks.
Important evaluation dimensions include:

- **Task success rate:** Did the agent actually achieve the objective?
- **Accuracy and factuality:** Were the intermediate and final outputs correct?
- **Efficiency:** How many steps and tool calls were required?
- **Cost:** How much computation, token usage, and API usage were consumed?
- **Latency:** How long did the complete task require?
- **Robustness:** Can the agent recover from failures?
- **Safety:** Did the agent avoid unauthorized or harmful actions?

# Key Takeaways

1. An **LLM is a model**; an **agent is a larger system**.
2. Agentic AI focuses on **goal-directed behaviour**.
3. Agents combine reasoning with **planning, memory, tools, actions, and feedback**.
4. The central loop is:

$$\textbf{Goal} \rightarrow \textbf{Reason} \rightarrow \textbf{Plan} \rightarrow \textbf{Act} \rightarrow \textbf{Observe} \rightarrow \textbf{Adapt}$$

5. Multi-agent systems distribute complex work across specialized agents.
6. Greater autonomy creates greater opportunities as well as greater risks.
7. Evaluation must measure not only response quality but also task success, efficiency, robustness, and safety.
8. Developer's Role: Review code changes, verify test results, and approve sensitive actions.
9. Agentic AI supports development; it does not replace developer judgment.
10. An agent is not just an LLM. It is a software system that combines a model with tools, state, execution logic, and operational controls.


תודה רבה
Hebrew
Ευχαριστώ
Greek
Danke
German
Спасибо
Russian
धन्यवादः
Sanskrit
Merci
French
ধন্যবাদ
Bangla
நன்றி
Tamil
شكراً
Arabic
ಧನ್ಯವಾದಗಳು
Kannada
Thank You
English
നന്ദി
Malayalam
Grazie
Italian
ధన్యవాదాలు
Telugu
આભાર
Gujarati
多謝
Traditional Chinese
ਧੰਨਵਾਦ
Punjabi
धन्यवाद
Hindi & Marathi
Gracias
Spanish
多谢
Simplified Chinese
https://sites.google.com/site/animeshchaturvedi07
Obrigado
Portuguese
ありがとうございました
Japanese
ขอบคุณ
Thai
감사합니다
Korean